PE-JV-003

Aula 06 - Observabilidade & Diagnóstico em Produção

Aquecimento

Kahoot

Revisão da Aula 05: Docker Multi-Stage, Imagens Distroless, Digest Imutável, Smoke Test na Esteira e Expand-Contract.

Aguarde as orientações do professor.

O Desafio de Engenharia

Na última aula, empacotamos o Checkout em imagem *Distroless* (sem shell) e discutimos estratégias de release sob **5.000 TPS** na Black Friday.

O incidente na Black Friday:
Minutos depois de a nova versão entrar em *Canary*, os painéis seguiam verdes: **latência média de 120 ms**, CPU abaixo de 40% e **zero HTTP 500**. Mesmo assim, a conversão desabou. Milhares de clientes reclamaram que o botão "Finalizar Pedido" ficava carregando até o aplicativo desistir por timeout.

O problema:
A imagem *Distroless* não tem `sh` e não há SSH nos servidores. Os logs são texto solto, sem correlação. Com o que existe, ninguém sabe onde o pedido trava. O que precisamos construir no software para conseguir investigar esse incidente e chegar à causa raiz (RCA)?

A Missão do Dia

No seu documento de System Design (Aula 06):

- Identificar o que falta no software para investigar este incidente e estruturar a **Root Cause Analysis (RCA / Postmortem Blameless)** com os sinais que a aplicação passa a expor.
- Desenhar a arquitetura dos **Três Pilares da Observabilidade** (Logs estruturados, Métricas e Tracing via OpenTelemetry com W3C `traceparent`), definindo estratégias de amostragem (*Sampling*) e conformidade com LGPD.
- Projetar painéis baseados nos **Métodos RED e USE** para capturar a experiência do cliente e a saturação interna de recursos.
- Definir a estratégia de testes de estresse com **k6 / Locust** para validar se a telemetria e os alertas respondem sob carga.

Sala de Aula Invertida: A Pesquisa

Acesse os materiais compartilhados e pesquise:

1. **Investigação e RCA Blameless:** O que a aplicação precisa expor para diagnosticar gargalos distribuídos sem acesso ao terminal, e como um postmortem *blameless* trata incidentes com mais de um fator contribuinte?
2. **OpenTelemetry e Correlação:** Como o W3C `traceparent` conecta requisições entre microsserviços, e como a amostragem (*Head* vs *Tail Sampling*) equilibra custo e visibilidade?
3. **Sinais RED vs USE:** Qual a diferença entre medir a saúde do serviço (Taxa, Erros, Duração) e medir a saturação de recursos (threads, pool de conexões)?
4. **Percentis vs Média & Cardinalidade:** Por que uma média de 120 ms pode esconder timeouts graves? E por que `user_id` jamais deve ser um *label* no Prometheus?
5. **Carga como Prova da Telemetria:** Como ferramentas de estresse (k6 / Locust) validam se painéis e alertas capturam anomalias antes do pico real?
6. **Dados de Cliente na Telemetria:** Pela LGPD, o que pode aparecer em logs e traces, e qual a diferença entre pseudonimizar e anonimizar?

O Desafio Arquitetural

No seu documento de System Design, fundamente suas decisões técnicas:

1. **Investigação em Produção e RCA Blameless:**
   - O que falta hoje no Checkout, no Pagamento e no Estoque para descobrir onde o pedido trava, e como mitigar o impacto enquanto a causa não aparece?
   - Quais hipóteses explicam o incidente, que sinal confirma ou descarta cada uma, e como estruturar o Postmortem *Blameless* separando o sintoma imediato dos fatores contribuintes sistêmicos?
2. **Arquitetura dos Três Pilares & OpenTelemetry:**
   - Qual o formato dos logs? Como propagar o contexto distribuído (`traceId`) entre Checkout, Pagamento e Estoque, em chamadas HTTP e eventos assíncronos?
   - Qual estratégia de amostragem (*Head vs. Tail Sampling*) equilibra custos sob 5.000 TPS sem descartar requisições com anomalias, e como garantir conformidade com a LGPD nos logs?
3. **Métricas RED/USE, Percentis e Validação sob Carga:**
   - Quais métricas externas (RED) e de saturação de recursos (USE) entram nos seus painéis, e como medir a duração para que o travamento de parte dos clientes não suma na média?
   - Quais *labels* são proibidos para evitar explosão de cardinalidade no TSDB? Como um teste de estresse (k6 / Locust) prova que a telemetria está pronta antes da Black Friday?

Debate Técnico

É hora de debater com o comitê de engenharia (Staff Engineer):

- **A Falácia da Média:** Por que nunca se deve tirar a média aritmética de percentis entre servidores e como histogramas resolvem a agregação?
- **Sintoma vs Saturação (RED & USE):** Como um recurso interno pode saturar sem disparar alerta de CPU nem erro 500?
- **Debug Container vs Telemetria:** Quando um *ephemeral debug container* ajuda? E quando só a telemetria responde?
- **A Armadilha da Cardinalidade:** O que acontece com o servidor de métricas quando alguém adiciona `order_id` ou CPF como label no Prometheus?
- **Amostragem em Larga Escala:** A 5.000 TPS, guardar todos os traces custa caro, e as requisições que travaram são uma fração pequena do tráfego. Se a aplicação sorteia na entrada quais traces guardar (*Head*), o que acontece com elas? E se o OTel Collector decide só depois que o trace termina (*Tail*), qual é o custo?
- **Carga e Prontidão de Telemetria:** Como k6 / Locust comprovam que os painéis e alertas disparam quando o sistema entra em estresse?
- **Contexto entre Threads e Eventos:** O que acontece com o `traceId` guardado no MDC quando o código troca de thread ou publica um evento da Saga?

Projeto Individual (System Design)

Cenário de Engenharia (Degradação de Checkout na Black Friday):

- **Arquitetura Distribuída:** Microsserviços de Checkout, Pagamento e Estoque operando sob **5.000 TPS** em containers *Distroless*.
- **Incidente:** Durante o *Canary*, latência média de 120 ms, CPU abaixo de 40% e zero HTTP 500, enquanto a conversão despenca e os clientes recebem timeout no fechamento da compra.
- **Restrições:** Imagens sem `sh`, servidores sem SSH, logs em texto livre sem correlação entre os serviços e dados de cliente sob LGPD.

Sua Missão:

- Responda às decisões do **Desafio Arquitetural** aplicadas diretamente ao cenário acima.
- Use o **Template da Aula 06** (disponível no LMS) e versione a entrega no repositório do seu projeto com commits atômicos e descritivos.

Laboratórios Práticos (Opcional)

O guia de laboratórios está disponível no LMS, com demos e ferramentas mantidas pelos próprios projetos:

- **Instrumentação Java com OpenTelemetry:** Guias oficiais do Java Agent e do Spring Boot Starter para instrumentar um serviço Spring e ver traces, métricas e logs correlacionados.
- **Grafana Play (Online, Sem Instalação):** Instância pública da Grafana (`play.grafana.org`) com painéis de latência e taxa de sucesso por serviço e fontes de dados Prometheus, Loki e Tempo para explorar métricas, logs e traces.
- **OpenTelemetry Demo (Astronomy Shop):** E-commerce distribuído mantido pelo projeto OpenTelemetry, via Docker Compose, com telemetria completa, gerador de carga e falhas ativadas por *feature flags*. A documentação pede 6 GB de RAM.
- **Grafana k6 & Locust (Simulação de Pico):** Roteiros práticos de *Spike Testing* para estressar a aplicação e validar a resposta dos painéis e alertas.

Próxima Aula: Segurança em APIs de Alto Volume

Nossa plataforma tem telemetria ponta a ponta e capacidade de diagnóstico em produção. Mas como blindar cada transação contra acessos indevidos e vazamento de dados de clientes?

Tópicos Conceituais:

- **Autenticação e Identidade:** Tokens JWT, assinatura criptográfica, validação de claims e scopes, SSO com OIDC/Keycloak e autenticação entre serviços com client credentials ou mTLS.
- **Autorização por Objeto (BOLA/IDOR):** Como impedir que um cliente acesse o recurso de outro?
- **OWASP Top 10:2025 e API Security Top 10:** Mapeamento das vulnerabilidades críticas em aplicações web e APIs, e padrões de defesa em profundidade.

Referências para Leitura Prévia:

- 📄 OWASP Top 10:2025
- 📄 OWASP API Security Top 10 (2023)
- 📄 RFC 8725: JSON Web Token BCP
- 📄 OAuth 2.0 (oauth.net)
- 📄 OpenID Connect: How It Works
- 📄 OWASP: Microservices Security Cheat Sheet

Referências da Aula 06 (Observabilidade & Diagnóstico)

Links oficiais para estudo e aprofundamento:

- 📄 **OpenTelemetry Overview:** What is OpenTelemetry?
- 📄 **Propagação de Contexto:** OpenTelemetry: Context Propagation
- 📄 **Especificação do `traceparent`:** W3C Trace Context
- 📄 **Estratégias de Amostragem:** OpenTelemetry: Sampling Concepts
- 📄 **Privacidade em Logs e Traces:** LGPD, Lei 13.709/2018 (Planalto)
- 📄 **Método RED:** The RED Method (Grafana Labs)
- 📄 **Método USE:** The USE Method (Brendan Gregg)
- 📄 **Boas Práticas de Métricas:** Prometheus: Metric Naming & Cardinality
- 📄 **Histogramas e Percentis:** Prometheus: Histograms & Summaries
- 📄 **Troubleshooting Sistemático:** Google SRE Book: Effective Troubleshooting
- 📄 **Cultura de Postmortem:** Google SRE Book: Postmortem Culture
- 📄 **Simulação de Carga com k6:** Spike Testing (Grafana k6)

Obrig.ADA